

M15 - Azure + Microsoft Fabric Data Engineering

Zero-to-Job-Ready Data Engineer 2026 | RC1 | 16 beginner-complete lessons

BEGINNER SUFFICIENCY RULE

This book teaches cloud/Fabric reasoning before UI clicks. UI labels can evolve; architecture, permissions, data contracts, Delta logic, run evidence and cost/reliability reasoning are the durable skill.

M15-L01 - Cloud Fundamentals for Data Engineers

Why this matters

Cloud services only help if you understand what is managed and what still belongs to you.

Start with the simple idea

Cloud platforms rent managed compute/storage/services instead of requiring you to own servers. “Managed” removes some infrastructure work but not responsibility for data correctness, permissions, cost or architecture.

Technical model

Region/location affects latency, residency and service availability.

Storage and compute can scale independently in many analytical architectures.

Managed services still require configuration, identity, monitoring and recovery decisions.

Code / configuration pattern

```
requirement = {'latency':'daily','data_size_gb':20,'sensitivity':'internal'}
```

Worked reasoning example

A 20GB nightly batch does not automatically need a constantly running high-end Spark cluster. A managed on-demand or capacity-backed service can execute the workload, then you measure run time/cost.

Common mistakes and debugging

- Treating cloud as infinite/free compute.
- Choosing services before requirements.
- Ignoring data residency/security policy.

Engineering decision

Start from workload, SLA, data sensitivity, team skill and cost envelope.

Before practice

P01 asks you to translate one local pipeline into cloud responsibilities.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L02 - Identity, IAM and Least Privilege

Why this matters

Most cloud incidents become worse when identities have more access than they need.

Start with the simple idea

Every human/service acts as an identity. Permissions should allow the required action on the smallest practical scope. Do not solve a permission error by making everyone admin.

Technical model

Microsoft Entra identities authenticate users/services; Fabric/workspace/item permissions govern access in the SaaS environment.

Separate human development access from unattended service identity when production automation is introduced.

Secrets/tokens are not source-code configuration.

Code / configuration pattern

bad

TOKEN='real-secret'

better: platform-managed connection / secret store / environment-specific configuration

Worked reasoning example

A pipeline that only reads source data and writes one lakehouse should not require organization-wide admin privileges. Grant the specific workspace/data permissions required and document ownership.

Common mistakes and debugging

- Hardcoding tokens.
- Using personal credentials for unattended production jobs.
- Granting admin to fix every 403/permission issue.

Engineering decision

Design identity and permission boundaries with the pipeline, not after deployment.

Before practice

G02 documents workspace/items and which identity needs which action.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L03 - Azure Storage Concepts and OneLake

Why this matters

OneLake becomes easier when you understand that cloud analytical storage is shared durable data, not a local C: drive.

Start with the simple idea

Object/lake storage keeps files/tables independently of one notebook process. OneLake is Fabric's unified organizational lake so multiple Fabric engines can work over shared governed data.

Technical model

OneLake is built on Azure Data Lake Storage foundations and provides a tenant-wide logical data lake.

Workspaces/lakehouses organize data ownership and access inside the unified storage model.

Open table formats such as Delta reduce engine lock-in and support Spark/SQL access.

Code / configuration pattern

OneLake / Workspace / Lakehouse / Files + Tables

Worked reasoning example

Raw CSV can land once in a lakehouse Files area; Spark transforms it into Delta tables; SQL consumers query the registered table without copying into an unrelated local file.

Common mistakes and debugging

- Thinking OneLake means one flat folder for the company.
- Duplicating data unnecessarily when a governed shortcut/shared access fits.
- Ignoring ownership across workspaces.

Engineering decision

Prefer one governed copy where practical, but keep clear domain/workspace ownership.

Before practice

G03 maps raw/curated paths and decides when copy vs shortcut/live access is appropriate.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L04 - Fabric Workspace and Item Model

Why this matters

Fabric combines many item types, so organization matters before data work begins.

Start with the simple idea

A workspace is a collaboration/deployment boundary containing items such as lakehouses, notebooks, pipelines and warehouses. Items have purpose; one giant miscellaneous workspace becomes hard to govern.

Technical model

Workspace roles/permissions affect who can create/use items.

Item names/environments should distinguish dev/test/prod or course/project contexts.

A lakehouse automatically has associated analytical access experiences such as the SQL analytics endpoint.

Code / configuration pattern

workspace: de-course-dev

items: lakehouse, notebook, pipeline

Worked reasoning example

Keep a course development workspace containing the lakehouse, notebook and pipeline that belong to one project. A production team may separate environments and deployment policies later.

Common mistakes and debugging

- Putting credentials in item names/descriptions.
- Mixing unrelated prod/test assets.
- Assuming workspace permission equals unrestricted data permission everywhere.

Engineering decision

Name and scope workspaces/items around ownership and lifecycle.

Before practice

G02 creates an item map and permission/evidence checklist.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L05 - OneLake Paths, Files, Tables and Shortcuts

Why this matters

The difference between raw files and managed/queryable tables matters in Fabric.

Start with the simple idea

Lakehouse Files can hold raw/source files. Tables represents managed/registered table data, commonly Delta. A shortcut can reference external/other OneLake data without copying it.

Technical model

Raw files are valuable for replay/audit but are not automatically an analytical table.

Delta tables add schema/transaction/table semantics learned in M13.

Shortcuts reduce copies but create dependency/governance/availability considerations.

Code / configuration pattern

Files/raw/orders/2026/08/orders.csv

Tables/bronze_orders

Worked reasoning example

A pipeline copies a daily CSV into Files/raw. A notebook validates and writes Bronze Delta. Analysts query the table through SQL endpoint. If a trusted master dataset already exists elsewhere, a shortcut may be better than another copy.

Common mistakes and debugging

- Assuming any file under Files appears as SQL table.
- Using shortcuts without understanding source permissions/lifecycle.
- Cleaning raw files destructively.

Engineering decision

Separate raw preservation from managed table contracts.

Before practice

G03/G04 create the planned hierarchy and table boundary.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L06 - Fabric Lakehouse and Delta Tables

Why this matters

Fabric lakehouse applies the M13 concepts in a managed cloud environment.

Start with the simple idea

A Fabric lakehouse combines lake storage with Delta table behavior. Spark engineers can transform data; SQL-oriented consumers can query Delta tables through the SQL analytics endpoint.

Technical model

Fabric lakehouse supports structured/unstructured storage and Delta-based tables.

Spark is the primary code-first transformation engine for lakehouse engineering.

Lakehouse and warehouse can coexist; SQL-first multi-table transactional needs may favor Warehouse.

Code / configuration pattern

```
df.write.format('delta').mode('overwrite').saveAsTable('bronze_orders')
```

Worked reasoning example

Use Spark to clean raw orders and save a Delta table. The table becomes visible for Spark/SQL consumption. You still validate grain/schema and rerun behavior rather than trusting successful UI status.

Common mistakes and debugging

- Calling Files folder a Delta table.
- Choosing lakehouse when every workload is SQL-first without evaluating Warehouse.
- Ignoring table grain because platform registration succeeded.

Engineering decision

Choose lakehouse vs warehouse by workload/tooling/data types and consumer needs.

Before practice

G04 writes/reads a Delta-style lakehouse table; G06 validates it via SQL.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L07 - Fabric Notebooks

Why this matters

Notebooks are productive only when code remains reproducible and evidence-driven.

Start with the simple idea

A notebook contains executable cells plus narrative. It is useful for exploration, ingestion and Spark transformation, but should not become an undocumented sequence that only works when cells are run in a secret order.

Technical model

Attach/select the intended lakehouse/environment before reading/writing.

Organize cells into configuration, input checks, transformations, validation and writes.

Production logic may later move to Spark job definitions or version-controlled packages where appropriate.

Code / configuration pattern

1 config -> 2 read -> 3 validate -> 4 transform -> 5 write -> 6 reconcile

Worked reasoning example

A notebook reads raw CSV, asserts required columns, calculates valid/rejected counts, writes Silver Delta and prints reconciliation. Another engineer can rerun from top with the same inputs.

Common mistakes and debugging

- Relying on stale variables from out-of-order cells.
- Hardcoding workspace-specific secrets.
- Writing outputs without control counts.

Engineering decision

Treat notebooks as code artifacts with deterministic run order and validation.

Before practice

G04/G05 use supplied notebook-source files in ordered sections.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L08 - Spark in Fabric

Why this matters

Managed Spark changes infrastructure work, not DataFrame correctness.

Start with the simple idea

The same PySpark concepts from M12 apply: schemas, transformations, joins, partitions and plans. Fabric manages cluster/session experience, but your code can still produce duplicates, skew, tiny files or incorrect joins.

Technical model

Use explicit schemas and built-in Spark functions where possible.

Measure row/key counts before/after joins and writes.

Fabric runtime/features may differ from local Spark; keep code portable when practical and record runtime evidence.

Code / configuration pattern

```
clean=(raw.dropDuplicates(['event_id']).filter('amount >= 0'))
```

Worked reasoning example

A cloud notebook finishing in 40 seconds is not proof the output is correct. Compare source/valid/rejected/Gold counts and inspect plan/layout for larger workloads.

Common mistakes and debugging

- Treating cloud compute as a substitute for validation.
- Collecting large data to driver.
- Ignoring partition/file layout.

Engineering decision

Reuse M12 performance/correctness habits in managed Spark.

Before practice

G05 transforms supplied sales data and records counts/schema.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L09 - Fabric Data Pipelines

Why this matters

Pipelines move/orchestrate data, but successful activity status is not a data-quality guarantee.

Start with the simple idea

A managed pipeline can copy data from source to lakehouse and coordinate activities. You still define source contract, destination, incremental behavior, retries and validation.

Technical model

Copy activity moves data; notebooks/other activities can transform or validate.

Pipeline parameters can support dates/paths rather than hardcoding one run.

Run history provides operational evidence but needs data reconciliation alongside it.

Code / configuration pattern

source -> copy raw -> notebook validate/transform -> quality gate

Worked reasoning example

A daily pipeline copies `sales\_20260827.csv` into raw. The copy succeeds but row count is zero because source file is empty. A downstream quality check must fail/alert rather than declaring the day successful.

Common mistakes and debugging

- Equating green pipeline with correct data.
- Hardcoding date/file path.
- Retrying deterministic schema errors forever.

Engineering decision

Design pipeline success criteria around business/data evidence.

Before practice

G07 creates a parameterized pipeline plan and evidence checklist.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L10 - SQL Analytics Endpoint and Warehouse Choice

Why this matters

Cloud DE must support consumers, not only Spark authors.

Start with the simple idea

Lakehouse Delta tables can be queried with T-SQL through the SQL analytics endpoint. Fabric Warehouse provides a more SQL-first warehouse experience. Know which surface you are using.

Technical model

Lakehouse SQL endpoint is oriented to querying registered Delta tables.

Warehouse favors structured SQL-first development and supports different transaction/workload capabilities.

Spark and SQL should agree on table grain/counts if they address the same data.

Code / configuration pattern

```
SELECT region, SUM(amount) AS revenue FROM gold_sales GROUP BY region;
```

Worked reasoning example

After Spark writes Gold, SQL endpoint returns category/region totals. If Spark count and SQL count disagree, investigate registration/table version/query logic rather than choosing the nicer number.

Common mistakes and debugging

- Assuming SQL endpoint can mutate every lakehouse use case like a normal OLTP database.
- Using a Warehouse because “SQL is familiar” without workload analysis.
- No cross-engine reconciliation.

Engineering decision

Choose the serving surface based on consumers and workload, while preserving one source of truth.

Before practice

G06 runs supplied validation SQL against the cloud table.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L11 - Medallion Architecture in Fabric

Why this matters

Bronze/Silver/Gold is useful only when each layer has an explicit contract.

Start with the simple idea

Bronze preserves source fidelity, Silver standardizes/trusts entities/events, Gold serves defined business grains. Fabric lakehouse makes these Delta layers natural, but the colors do not define quality by themselves.

Technical model

Bronze should support replay/audit.

Silver applies schema, dedupe, quality and current/history rules.

Gold expresses consumer-ready grain/metrics.

Code / configuration pattern

```
raw -> bronze_sales -> silver_sales -> gold_region_sales
```

Worked reasoning example

An invalid negative sale remains auditable in raw/Bronze, is rejected/quarantined from trusted Silver, and therefore does not contaminate Gold revenue. Reconciliation explains the difference.

Common mistakes and debugging

- Deleting bad raw records.
- Letting duplicates pass because Gold “will aggregate them away.”
- Building Gold before defining grain/consumer.

Engineering decision

Document layer input/output grain, quality and reject behavior.

Before practice

G08 builds the incremental medallion plan and validates rerun counts.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L12 - Incremental Loads and MERGE in Cloud

Why this matters

Cloud scale makes unnecessary full reloads expensive and slow.

Start with the simple idea

Daily changes should update/insert deterministically when source semantics support it. M08/M13 watermark/idempotency/MERGE concepts carry directly into Fabric.

Technical model

Use a stable business/event key plus deterministic precedence.

Advance control state only after successful validated write.

Replay the same incremental batch and verify logical counts stay stable.

Code / configuration pattern

```
# pseudocode
target.alias('t').merge(changes.alias('s'),'t.sale_id=s.sale_id').whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
```

Worked reasoning example

Initial load has S1-S5. Incremental batch updates S2 and inserts S6/S7. Final key count becomes 7, and replaying the same batch stays 7.

Common mistakes and debugging

- Appending updates.
- Using ingestion time when source updates can arrive late.
- Running full reload because it is easier.

Engineering decision

Prefer incremental patterns only when their correctness can be proven and repaired.

Before practice

G08 performs a local control simulation and maps it to Fabric Delta MERGE.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L13 - Permissions, Connections and Secrets

Why this matters

Cloud pipelines often fail security review when credentials are treated like normal strings.

Start with the simple idea

Use managed platform connections/identities and least privilege. Keep environment-specific connection details out of shared code where possible and never commit real secrets.

Technical model

Separate authentication from transformation logic.

Grant read/write scope only where the job needs it.

Review workspace/item/data access when sharing notebooks or lakehouses.

Code / configuration pattern

code references logical configuration; credential is platform-managed

Worked reasoning example

A developer can edit notebook code but the production pipeline executes through an approved service identity/connection. Revoking the developer account should not break production.

Common mistakes and debugging

- Personal access tokens in notebook cells.
- Broad workspace admin role for a read-only consumer.
- Sharing lakehouse without reviewing sensitive columns.

Engineering decision

Treat identity/data access as part of architecture and release review.

Before practice

G09 creates a least-privilege matrix and secret scan checklist.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L14 - Monitoring, Reliability and Cost Awareness

Why this matters

A managed service still needs operational ownership.

Start with the simple idea

Monitor run status, duration, input/output counts, errors, freshness and compute/capacity behavior. Cost awareness means understanding which repeated/large Spark/pipeline operations consume resources.

Technical model

Notebook/pipeline run history gives execution evidence.

Data-quality/freshness metrics provide business health evidence.

Cloud cost/performance optimization begins with measured workload, not random settings.

Code / configuration pattern

run_id, start, end, source_rows, loaded_rows, rejected_rows, status

Worked reasoning example

A notebook runtime doubles but row volume is unchanged. Inspect Spark plan/file layout/capacity contention before buying more capacity. A zero-row pipeline should alert even if the activity technically succeeded.

Common mistakes and debugging

- Watching only UI “Succeeded.”
- Leaving expensive test loops running.
- Optimizing tiny lab datasets as if production scale.

Engineering decision

Monitor both platform execution and data outcomes.

Before practice

G09 builds a monitoring/cost checklist and incident hypothesis.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L15 - Git, Deployment and Environment Separation

Why this matters

Cloud notebooks and pipelines become safer when changes are versioned and promoted deliberately.

Start with the simple idea

Development, test and production should not be one mutable workspace with manual edits. Source control and deployment practices help reproduce releases and review changes.

Technical model

Keep code/config artifacts under version control where supported.

Separate dev/test/prod parameters and identities.

Deployment should include validation and rollback/recovery thinking.

Code / configuration pattern

main -> reviewed change -> dev validation -> test -> controlled prod promotion

Worked reasoning example

A notebook change modifies Gold grain. Git review catches the contract change; test workspace validates counts before production promotion. Direct production editing would hide history and increase rollback risk.

Common mistakes and debugging

- Editing production notebook directly.
- Same secret/connection across environments without control.
- Deploying without data contract checks.

Engineering decision

Cloud source control is part of DataOps, not optional portfolio decoration.

Before practice

P09 asks for a three-environment promotion design.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

M15-L16 - End-to-End Fabric Data Engineering Case

Why this matters

Job readiness requires connecting storage, Spark, pipelines, SQL, security and evidence.

Start with the simple idea

A complete Fabric pipeline lands source data in OneLake/lakehouse, preserves raw, transforms trusted Delta tables with Spark, builds Gold outputs, schedules movement/orchestration, exposes SQL-facing data, and records quality/run evidence.

Technical model

Architecture must define identities, layer contracts, incremental behavior and consumers.

One successful run is insufficient: perform a rerun/incremental update and failure/recovery check.

Cloud implementation should be explainable in AWS/GCP terms even if Fabric is the primary hands-on stack.

Code / configuration pattern

source -> Fabric Pipeline -> OneLake/Lakehouse raw -> Spark Bronze/Silver/Gold -> SQL endpoint -> consumer

Worked reasoning example

A sales pipeline ingests historical data, merges a daily update, produces Gold region totals, queries them via SQL endpoint, then reruns the same update without increasing distinct sale IDs. Pipeline and notebook run evidence plus permissions are documented.

Common mistakes and debugging

- Architecture diagram that omits identity/quality/recovery.
- Using streaming where daily batch is sufficient.
- Treating cloud screenshots as proof without counts/results.

Engineering decision

A cloud project is complete when it is correct, repeatable, secure enough for its scope, observable and explainable.

Before practice

M15 Module Practical integrates the entire module; fresh retest changes domain and key structure.

Explain it without notes

Explain the design in 3-5 sentences, name the Fabric/OneLake/Spark/Pipeline/SQL/security evidence you would inspect, and state one cloud mistake that could cause wrong data, excess cost, over-permission, or an unrecoverable deployment.

Cloud concept translation card

Concept	Azure/Fabric primary	AWS idea	GCP idea
Object/lake storage	OneLake / ADLS concepts	S3	Cloud Storage
Warehouse	Fabric Warehouse	Redshift	BigQuery
Spark	Fabric Spark / Databricks adjacency	EMR / Databricks	Dataproc / Databricks
Orchestration	Fabric Pipelines / ADF concepts	Glue/MWAA/Step Functions depending design	Composer/Workflows depending design